# UF1 – Promesas y Async/Await

### JavaScript – 2º DAW – Desarrollo Web en Entorno Cliente

**Prof. Sergio Martínez**

---

## 1️⃣ Introducción a la programación asíncrona

JavaScript es un lenguaje de programación **monohilo**, lo que significa que ejecuta una sola tarea a la vez.
Sin embargo, muchas operaciones —como la carga de un archivo, una consulta a una API o la espera de una respuesta del servidor— tardan tiempo, y **no podemos permitir que el navegador se quede bloqueado** mientras tanto.

Para resolver esto, JavaScript utiliza **operaciones asíncronas**, que permiten que una tarea se inicie ahora y se complete más adelante, sin detener el flujo principal del programa.

---

### 💡 Ejemplos comunes de acciones asíncronas

- Cargar una imagen o script externo.
- Esperar una respuesta de una API (petición HTTP).
- Temporizadores (`setTimeout`, `setInterval`).
- Eventos del usuario (clics, teclas, movimientos del ratón).

Todas ellas se gestionan de forma asíncrona mediante un sistema interno conocido como el **event loop**.

---

### 🧩 Ejemplo básico con `setTimeout`

```
console.log("Inicio");
setTimeout(() => console.log("Mensaje retrasado"), 2000);
console.log("Fin");
```

### 📘 Salida:

```
Inicio
Fin
Mensaje retrasado
```

👉 Aunque el `setTimeout` está en el medio del código, **se ejecuta al final**, porque se programa para más adelante.
Este comportamiento es lo que llamamos **asincronía**.

---

# 2️⃣ Callbacks: la primera solución

Antes de las promesas y `async/await`, JavaScript usaba **callbacks** para manejar tareas asíncronas.
Un callback es simplemente una **función que se pasa como argumento** a otra función, para que se ejecute cuando la tarea haya terminado.

---

## 🧩 Ejemplo con callback

```
function loadScript(src, callback) {
  let script = document.createElement('script');
  script.src = src;
  script.onload = () => callback(null, script);
  script.onerror = () => callback(new Error("Error al cargar"));
  document.head.append(script);
}

loadScript("script1.js", function(err, script) {
  if (err) console.error(err);
  else console.log("Script cargado:", script.src);
});
```

💡 **Ventaja:** Podemos ejecutar algo cuando el script termina de cargarse.
⚠️ **Desventaja:** Si queremos cargar varios scripts o hacer varias tareas en orden, el código se vuelve un desastre.

---

## ⚠️ El “infierno de callbacks”

Cuando las tareas se encadenan una dentro de otra, el código empieza a crecer **hacia la derecha**, dificultando su lectura y mantenimiento.

```
loadScript("1.js", function() {
  loadScript("2.js", function() {
    loadScript("3.js", function() {
      console.log("Todos los scripts cargados");
    });
  });
});
```

📉 Este estilo es lo que se conoce como **callback hell** o **pirámide infernal**.
Cada nueva operación depende de la anterior, lo que hace el código largo, confuso y difícil de depurar.

---

## 💡 Ejemplo con manejo de errores

En las funciones basadas en callback, se utiliza una convención llamada **error-first**, donde el primer parámetro es el error (si lo hay), y el segundo es el resultado:

```
function operacionAsincrona(callback) {
  let exito = Math.random() > 0.5;
  if (exito) callback(null, "Operación correcta");
  else callback(new Error("Algo falló"));
}

operacionAsincrona((err, resultado) => {
  if (err) console.error("Error:", err.message);
  else console.log(resultado);
});
```

💬 Este sistema funciona, pero **no escala bien** cuando hay muchas operaciones encadenadas o múltiples niveles de dependencia.

---

## 📊 Resumen del problema

| Problema | Consecuencia |
|---|---|
| Múltiples callbacks anidados | Código ilegible |
| Manejo manual de errores | Difícil de mantener |
| Pérdida de contexto | Variables y flujo complicados |
| Poca reutilización | Callbacks específicos por tarea |

---

## 💡 Conclusión:

Los callbacks fueron el primer intento de manejar la asincronía, pero a medida que las aplicaciones crecieron, se hicieron insuficientes.
De ahí nacen las **Promesas**, un nuevo modelo más limpio y potente para gestionar el código asíncrono.

# Bloque 2 – Promesas: el nuevo modelo de asincronía

## 1️⃣ Qué es una Promesa

Una **Promesa** es un **objeto especial de JavaScript** que representa el **resultado futuro de una operación asíncrona**.
Podemos imaginarla como un *contrato*: el programa promete que en algún momento habrá un resultado (éxito ✅ o error ❌).

Cada promesa tiene tres **estados posibles**:

| Estado | Significado |
|---|---|
| `pending` | La operación sigue en curso (pendiente). |
| `fulfilled` | La operación terminó correctamente. |
| `rejected` | La operación falló y devolvió un error. |

Cuando una promesa cambia de `pending` a cualquiera de los otros dos, **queda "resuelta"** y ya no puede volver atrás.

---

## 2️⃣ Creación de una Promesa

La sintaxis básica usa el **constructor `Promise`**, que recibe una función *ejecutora* con dos parámetros:

```
let promesa = new Promise((resolve, reject) => {
  // 1. Ejecutamos alguna tarea que tarda tiempo
  setTimeout(() => {
    let exito = true;
    if (exito) resolve("¡Operación completada!");
    else reject("Ha ocurrido un error");
  }, 1000);
});
```

💬 La función ejecutora se ejecuta **inmediatamente** al crear la promesa.

- Si la tarea sale bien, se llama a `resolve(valor)`.
- Si algo falla, se llama a `reject(error)`.

---

## 3️⃣ Consumidores: `.then()` y `.catch()`

Para reaccionar al resultado de una promesa, usamos sus **métodos principales**:

| Método | Cuándo se ejecuta |
| --- | --- |
| `.then(función)` | Cuando la promesa se **cumple correctamente**. |
| `.catch(función)` | Cuando la promesa **falla o se rechaza**. |

```
promesa
  .then(resultado => console.log("Resultado:", resultado))
  .catch(error => console.error("Error:", error));
```

💡 El método `.then()` recibe el valor pasado a `resolve()`.
Si la promesa falla, el control pasa al `.catch()`.

---

# 4️⃣ Ejemplo práctico

```
function comprobarUsuario(nombre) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (nombre === "Sergio") resolve("Acceso concedido ✅");
      else reject("Usuario no autorizado ❌");
    }, 1500);
  });
}

comprobarUsuario("Sergio")
  .then(msg => console.log(msg))
  .catch(err => console.error(err));
```

📘 **Salida:**

```
Acceso concedido ✅
```

Si cambiamos el nombre por otro valor, el flujo saltará al `.catch()`:

```
Usuario no autorizado ❌
```

---

# 5️⃣ El método `.finally()`

Al igual que en un bloque `try…catch…finally`, este método se ejecuta **siempre**, haya éxito o error.
Sirve para liberar recursos, ocultar indicadores de carga, etc.

```
comprobarUsuario("Alex")
  .then(msg => console.log(msg))
  .catch(err => console.error(err))
  .finally(() => console.log("Operación completada ✔"));
```

📘 Salida:

```
Usuario no autorizado ❌
Operación completada ✔
```

💡 `.finally()` no recibe el resultado ni el error; su función es **limpiar** o **cerrar** procesos.

---

# 6️⃣ Encadenamiento de promesas

Podemos **encadenar varios** `.then()`, pasando el resultado de uno al siguiente.

```
new Promise(resolve => setTimeout(() => resolve(2), 1000))
  .then(num => num * 2)   // devuelve 4
  .then(num => num + 3)   // devuelve 7
  .then(num => console.log("Resultado final:", num));
```

### 📘 Salida:

```
Resultado final: 7
```

Cada `.then()` devuelve una nueva promesa, lo que permite construir flujos ordenados y fáciles de seguir.

---

### 💡 Resumen del bloque:

Las promesas sustituyen a los callbacks ofreciendo un sistema claro, encadenable y con manejo de errores integrado.
`.then()` procesa los éxitos, `.catch()` los fallos y `.finally()` siempre limpia al final.

---

# Bloque 3 – Manejo de errores en Promesas

Las promesas permiten gestionar los errores de forma **limpia y controlada**, sin tener que anidar múltiples callbacks.
Todo error o rechazo dentro de una promesa puede **capturarse y tratarse fácilmente** con `.catch()`.

---

## 1️⃣ El método `.catch()`

El método `.catch()` se ejecuta cuando una promesa es **rechazada** o cuando ocurre un error dentro de un `.then()`.

```
fetch("https://api.inventada123.com/datos")
  .then(response => response.json())  // Esto lanzará un error
  .catch(error => console.error("Error al obtener datos:", error));
```

💡 El `.catch()` no necesita estar inmediatamente después del `.then()`;
JavaScript recorrerá toda la cadena hasta encontrarlo.

---

## 2️⃣ Errores dentro de `.then()`

Si un error ocurre dentro de un `.then()`, se interpreta como un **rechazo** y se envía automáticamente al `.catch()` más cercano.

```
new Promise((resolve) => resolve("OK"))
  .then(() => {
    throw new Error("Algo falló dentro del then");
  })
  .catch(err => console.error("Error capturado:", err.message));
```

📘 **Salida:**

```
Error capturado: Algo falló dentro del then
```

💬 Esto significa que **no necesitamos escribir try/catch dentro de cada then**, ya que el propio motor de Promesas lo gestiona.

---

## 3️⃣ Relanzar (rethrow) un error

Podemos capturar un error en un `.catch()` y volver a lanzarlo para que otro bloque más arriba lo maneje.

```
new Promise((resolve, reject) => {
  reject(new Error("Error inicial"));
})
.catch(error => {
  console.warn("Primer manejo:", error.message);
  throw error; // re-lanzamos
})
.catch(error => {
  console.error("Error final:", error.message);
});
```

📘 **Salida:**

```
Primer manejo: Error inicial
Error final: Error inicial
```

💡 Esto se usa cuando una parte del código solo puede registrar o analizar el error, pero no resolverlo.

---

# 4️⃣ Errores no manejados (unhandled rejections)

Cuando una promesa falla y no existe ningún `.catch()` asociado, el navegador genera una advertencia:

```
new Promise((_, reject) => reject("Fallo grave"));
// ⚠️ Unhandled Promise Rejection
```

👉 Podemos capturar todos los errores globales con el evento `unhandledrejection`:

```
window.addEventListener("unhandledrejection", event => {
  console.error("Rechazo no manejado:", event.reason);
});
```

💬 En Node.js se usa `process.on('unhandledRejection', …)` para el mismo propósito.

---

# 5️⃣ Encadenamiento con errores mixtos

```
fetch("https://api.github.com/users/octocat")
  .then(resp => resp.json())
  .then(user => {
    if (!user.login) throw new Error("Usuario no válido");
    return user;
  })
  .then(user => console.log("Usuario cargado:", user.login))
```

```
  .catch(err => console.error("Error detectado:", err.message))
  .finally(() => console.log("Proceso finalizado"));
```

📘 **Salida:**

```
Usuario cargado: octocat
Proceso finalizado
```

o bien

```
Error detectado: Usuario no válido
Proceso finalizado
```

💡 `.catch()` captura cualquier excepción en la cadena anterior,
y `.finally()` se ejecuta siempre, ocurra lo que ocurra.

---

# 6️⃣ Buenas prácticas con errores en Promesas

✅ Coloca siempre un `.catch()` al final de las cadenas.
✅ Usa mensajes de error descriptivos (`reject(new Error("…"))`).
✅ Evita anidar `.then()` innecesarios; encadena en vertical.
✅ Utiliza `.finally()` para limpiar o cerrar conexiones.

---

**Conclusión del bloque:**

Las promesas incorporan un sistema de manejo de errores potente y coherente.
Con solo un `.catch()` al final puedes controlar cualquier fallo dentro de toda la cadena.

# Bloque 4 – Promise API avanzada

La clase `Promise` incluye varios **métodos estáticos** que permiten trabajar con varias promesas al mismo tiempo.
Son fundamentales cuando tenemos que **sincronizar tareas paralelas**, como cargar varios recursos, consultar múltiples APIs o esperar diferentes operaciones antes de continuar.

---

## 1️⃣ Promise.all()

Ejecuta varias promesas **en paralelo** y devuelve una **nueva promesa** que se resuelve cuando **todas** han finalizado con éxito.
Si alguna de ellas falla, toda la operación se rechaza.

```
Promise.all([
  fetch("https://api.github.com/users/octocat"),
  fetch("https://api.github.com/users/torvalds"),
  fetch("https://api.github.com/users/mojombo")
])
  .then(respuestas => Promise.all(respuestas.map(r => r.json())))
  .then(usuarios => console.log("Usuarios:", usuarios.map(u =>
u.login)))
  .catch(error => console.error("Error en Promise.all:", error));
```

📘 **Salida:**

```
Usuarios: ["octocat", "torvalds", "mojombo"]
```

⚠️ Si una promesa falla, el `.catch()` se ejecuta y las demás se ignoran.
💡 Úsalo cuando todas las tareas sean **imprescindibles** para continuar.

---

## 2️⃣ Promise.allSettled()

Devuelve una promesa que **siempre se resuelve**, aunque algunas operaciones fallen.
El resultado es un array con el estado (`fulfilled` o `rejected`) de cada promesa.

```
Promise.allSettled([
  Promise.resolve("✅ Correcta"),
  Promise.reject("❌ Falló"),
  Promise.resolve("✅ Otra correcta")
]).then(resultados => console.table(resultados));
```

📘 **Salida:**

| status | value / reason |
| --- | --- |
| fulfilled | "✅ Correcta" |
| rejected | "❌ Falló" |
| fulfilled | "✅ Otra correcta" |

💡 Ideal cuando necesitas **procesar todo el conjunto**, incluso si algunas tareas no se completan correctamente.

---

# 3️⃣ Promise.race()

Devuelve el resultado de **la primera promesa que termina**, sin importar si fue un éxito o un error.

```
Promise.race([
  new Promise(resolve => setTimeout(() => resolve("🏁 Gana A"), 1000)),
  new Promise(resolve => setTimeout(() => resolve("Gana B"), 2000))
])
  .then(resultado => console.log(resultado)); // "🏁 Gana A"
```

💡 Muy útil para implementar **timeouts** o **competencias** entre tareas (por ejemplo, detener una petición si tarda demasiado).

---

# 4️⃣ Promise.any()

Devuelve el resultado de **la primera promesa que se cumple** con éxito.
Ignora las que se rechazan, a menos que todas fallen.

```
Promise.any([
  Promise.reject("Servidor 1 caído"),
  Promise.resolve("Servidor 2 activo"),
  Promise.reject("Servidor 3 caído")
])
  .then(resultado => console.log(resultado)) // "Servidor 2 activo"
  .catch(error => console.error("Todas fallaron:", error));
```

⚠️ Si todas las promesas se rechazan, genera un **AggregateError** con todos los errores almacenados.

---

# 5️⃣ Promise.resolve() y Promise.reject()

Sirven para **crear manualmente promesas resueltas o rechazadas**.
Se usan para mantener compatibilidad o adaptar valores normales a un flujo de promesas.

```
Promise.resolve("Listo").then(console.log);  // "Listo"
Promise.reject("Error forzado").catch(console.error); // "Error forzado"
```

💡 Muy práctico cuando una función debe **devolver siempre una promesa**, aunque no sea estrictamente asíncrona.

---

# 6️⃣ Cuadro comparativo rápido

| Método | Espera todas | Falla si una falla | Resultado |
|---|---|---|---|
| `Promise.all()` | ✅ | ✅ | Array de valores |
| `Promise.allSettled()` | ✅ | ❌ | Array con estados |
| `Promise.race()` | ❌ | ⚠️ depende del primero | Valor o error |
| `Promise.any()` | ❌ | ❌ | Primer éxito |
| `Promise.resolve()` | — | — | Promesa cumplida |
| `Promise.reject()` | — | — | Promesa rechazada |

---

💡 **Conclusión del bloque:**

La API estática de Promises permite ejecutar y coordinar múltiples tareas asíncronas de manera eficaz.
Elegir el método adecuado depende de si necesitas **todas las respuestas**, **solo la primera** o **la primera exitosa**.

# Bloque 5 – Promisificación

## 1️⃣ ¿Qué es la promisificación?

💡 **Definición:**
La *promisificación* es el proceso de **convertir una función que usa callbacks en una función que devuelve una Promesa**.

Esto permite utilizarla fácilmente con `.then()`, `.catch()` o incluso `async/await`, haciendo el código más claro y reutilizable.

📘 Es muy común en Node.js y en librerías antiguas del navegador que aún usan callbacks.

---

## 2️⃣ Ejemplo clásico con callback

Supongamos una función que carga un script y recibe un callback:

```
function loadScript(src, callback) {
  let script = document.createElement('script');
  script.src = src;
  script.onload = () => callback(null, script);
  script.onerror = () => callback(new Error("Error al cargar el
script"));
  document.head.append(script);
}
```

Uso clásico:

```
loadScript("main.js", function(err, script) {
  if (err) console.error(err);
  else console.log("Script cargado:", script.src);
});
```

⚠️ Inconvenientes:

- Necesitamos escribir lógica de éxito y error en el mismo lugar.
- No podemos encadenar acciones fácilmente.
- El código crece en profundidad si hay más operaciones.

---

## 3️⃣ Convirtiendo a Promesa manualmente

Podemos crear una **versión moderna** que devuelva una promesa:

```
function loadScriptPromise(src) {
```

```
  return new Promise((resolve, reject) => {
    let script = document.createElement('script');
    script.src = src;
    script.onload = () => resolve(script);
    script.onerror = () => reject(new Error("Error al cargar el
script"));
    document.head.append(script);
  });
}
```

Uso:

```
loadScriptPromise("main.js")
  .then(script => console.log("Script cargado:", script.src))
  .catch(err => console.error(err));
```

💡 Ventajas:

- Código más limpio y lineal.
- Compatible con `async/await`.
- Reutilizable en diferentes contextos.

---

# 4️⃣ Promisificar de forma genérica

Podemos crear una función que *promisifique* cualquier función de estilo callback:

```
function promisify(func) {
  return function(...args) {
    return new Promise((resolve, reject) => {
      func(...args, (err, result) => {
        if (err) reject(err);
        else resolve(result);
      });
    });
  };
}
```

---

## 🧩 Ejemplo de uso:

```
function getData(callback) {
  setTimeout(() => callback(null, "Datos cargados correctamente"),
1000);
}

const getDataPromise = promisify(getData);

getDataPromise()
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

## 📘 Salida:

```
Datos cargados correctamente
```

---

## 5️⃣ Versión avanzada (varios argumentos)

Algunas funciones de callback devuelven **más de un valor de resultado**, por ejemplo: `callback(err, resultado1, resultado2)`.

Podemos adaptar nuestra función `promisify` para manejarlo:

```
function promisifyMany(func) {
  return function(...args) {
    return new Promise((resolve, reject) => {
      func(...args, (err, ...results) => {
        if (err) reject(err);
        else resolve(results);
      });
    });
  };
}
```

Uso:

```
function dividir(a, b, callback) {
  if (b === 0) callback(new Error("División por cero"));
  else callback(null, a / b, "Operación completada");
}

const dividirPromise = promisifyMany(dividir);

dividirPromise(10, 2)
  .then(([resultado, mensaje]) => console.log(resultado, mensaje))
  .catch(err => console.error(err.message));
```

### 📘 Salida:

```
5 Operación completada
```

---

### 💡 Conclusión del bloque:

La promisificación permite migrar código antiguo basado en callbacks al modelo moderno de Promesas.
Es un paso clave para combinarlo con `async/await` y mantener un estilo uniforme y limpio en toda la aplicación.

# Bloque 6 – Async / Await

Las palabras clave `async` y `await` son la evolución natural de las Promesas.
Permiten escribir código **asíncrono que parece síncrono**, es decir, se lee de arriba abajo de manera clara, sin necesidad de encadenar `.then()`.

---

## 1️⃣ ¿Qué hace `async`?

Una función declarada con `async` **devuelve siempre una Promesa**.

```
async function saludar() {
  return "Hola mundo";
}

saludar().then(mensaje => console.log(mensaje)); // "Hola mundo"
```

💡 Si la función devuelve un valor normal, JavaScript lo envuelve automáticamente en una promesa resuelta.

Equivalente a:

```
function saludar() {
  return Promise.resolve("Hola mundo");
}
```

---

## 2️⃣ ¿Qué hace `await`?

`await` **pausa** la ejecución dentro de una función `async` hasta que la promesa se resuelva.
De esta forma podemos trabajar con los resultados directamente, sin usar `.then()`.

```
async function esperarMensaje() {
  console.log("Esperando...");
  let mensaje = await new Promise(resolve =>
    setTimeout(() => resolve("¡Listo!"), 2000)
  );
  console.log(mensaje);
}

esperarMensaje();
```

📘 **Salida:**

```
Esperando...
¡Listo!
```

💡 Mientras la función espera, JavaScript puede continuar ejecutando otras tareas: **no se bloquea el hilo principal**.

---

## 3️⃣ Ejemplo práctico con `fetch`

Reescribamos una cadena de promesas usando `async/await`:

```
async function mostrarUsuario() {
  try {
    const respuesta = await
fetch("https://api.github.com/users/octocat");
    const datos = await respuesta.json();
    console.log("Usuario:", datos.login);
  } catch (error) {
    console.error("Error al cargar el usuario:", error);
  } finally {
    console.log("Petición completada");
  }
}

mostrarUsuario();
```

💬 Aquí la función realiza **dos esperas (`await`)**:

- una para `fetch()` (la petición HTTP),
- otra para `respuesta.json()` (procesar la respuesta).

---

## 4️⃣ Manejo de errores con `try...catch`

Dentro de una función `async`, podemos usar `try...catch` igual que en código normal. Cualquier error lanzado dentro (o dentro de una promesa esperada) será capturado.

```
async function cargarDatos() {
  try {
    const resp = await fetch("https://api.inventada123.com");
    const datos = await resp.json();
    console.log(datos);
  } catch (error) {
    console.error("Error al obtener datos:", error);
  }
}

cargarDatos();
```

📘 Si la URL no existe, el flujo saltará directamente al bloque `catch`.

---

## 5️⃣ Combinando `await` con `Promise.all()`

Podemos usar `await` junto con `Promise.all()` para ejecutar **varias tareas en paralelo**.

```
async function cargarTodo() {
  const [user, repos] = await Promise.all([
    fetch("https://api.github.com/users/octocat").then(r => r.json()),
    fetch("https://api.github.com/users/octocat/repos").then(r =>
r.json())
  ]);

  console.log("Usuario:", user.login);
  console.log("Repositorios:", repos.length);
}

cargarTodo();
```

💡 De esta forma, las dos peticiones se ejecutan **al mismo tiempo**,
y el programa espera hasta que ambas hayan terminado.

---

## 6️⃣ `await` solo dentro de funciones `async`

No podemos usar `await` directamente fuera de una función `async`:

```
// ❌ Esto da error
let datos = await fetch("https://api.github.com/users/octocat");
```

⚠️ Este error ocurre porque `await` necesita un contexto `async`.
Para usarlo en el nivel superior, debe estar dentro de una función `async` o usarse en un entorno moderno que lo permita (por ejemplo, módulos ECMAScript con *top-level await*).

---

## 7️⃣ `async` y `await` con funciones flecha

También funcionan con funciones flecha:

```
const obtenerUsuario = async () => {
  const respuesta = await
fetch("https://api.github.com/users/octocat");
  const datos = await respuesta.json();
  console.log(datos.login);
};

obtenerUsuario();
```

💡 `async` se coloca **antes** de los paréntesis, no dentro.

---

## 8️⃣ Ventajas de `async/await`

| Ventaja | Descripción |
|---|---|
| ✅ Código más legible | Se lee de arriba abajo, como si fuera síncrono. |
| ✅ Manejo de errores fácil | Usa `try...catch` tradicional. |
| ✅ Combina con Promises | Puedes usar `await` sobre cualquier promesa. |
| ✅ Menos `.then()` encadenados | Elimina la "piramide de promesas". |

---

## 9️⃣ Comparación práctica

### 🔶 Versión con Promesas

```
fetch("https://api.github.com/users/octocat")
  .then(r => r.json())
  .then(data => console.log(data.login))
  .catch(e => console.error(e));
```

### 🔷 Versión con Async/Await

```
async function ejemplo() {
  try {
    const r = await fetch("https://api.github.com/users/octocat");
    const data = await r.json();
    console.log(data.login);
  } catch (e) {
    console.error(e);
  }
}
ejemplo();
```

📘 Misma funcionalidad, pero más clara, directa y mantenible.

---

## 1️⃣0️⃣ Buenas prácticas con `async/await`

✅ Usa `try...catch` siempre que pueda fallar algo.
✅ Evita ejecutar `await` dentro de bucles —usa `Promise.all()` si puedes.
✅ Encapsula tareas repetitivas en funciones `async`.
✅ Añade `.finally()` si necesitas limpiar recursos o mostrar mensajes finales.

---

💡 **Conclusión del bloque:**

`async/await` es la forma moderna y recomendada de trabajar con operaciones asíncronas en JavaScript.
Permite escribir código más claro, predecible y mantenible.

---

# Bloque 7 – Resumen general y esquema visual

---

## 1️⃣ De los callbacks a las promesas

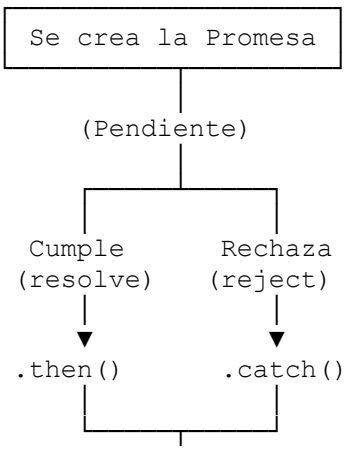
💬 En los inicios de JavaScript, las tareas asíncronas se manejaban con **callbacks**, pero rápidamente se volvieron difíciles de mantener: errores anidados, pérdida de contexto y código ilegible.

| Enfoque | Características | Inconvenientes |
|---|---|---|
| **Callbacks** | Pasas una función como argumento que se ejecuta después de la tarea. | Código en cascada (*callback hell*). |
| **Promesas** | Encapsulan una tarea y su resultado futuro. | Código más limpio, pero con varios `.then()`. |
| **Async/Await** | Permite escribir promesas con sintaxis secuencial. | Necesita entorno moderno (ES8+). |

💡 **Conclusión:** cada paso ha hecho el código asíncrono más simple, legible y seguro.

---

## 2️⃣ Flujo visual de una Promesa



```
        ▼
    .finally()
```

💬 Este flujo muestra que una promesa **solo cambia una vez de estado**,
y que `.catch()` y `.finally()` garantizan un manejo controlado del resultado.

---

## 3️⃣ Flujo visual de `async/await`

```
async function tarea() {
  try {
    const resultado = await operacionAsincrona();
    console.log(resultado);
  } catch (error) {
    console.error(error);
  } finally {
    console.log("Tarea finalizada");
  }
}
```

📘 **Funcionamiento:**
1️⃣ La palabra `async` hace que la función devuelva una promesa.
2️⃣ `await` pausa la ejecución hasta que la promesa termina.
3️⃣ Los errores se manejan con `try...catch`.
4️⃣ `finally` se ejecuta siempre, ocurra lo que ocurra.

---

## 4️⃣ Cuadro comparativo: Promesas vs Async/Await

| Aspecto | Promesas (`.then()`) | Async/Await |
|---|---|---|
| Sintaxis | Encadenada, con `.then()` y `.catch()` | Secuencial, más limpia |
| Legibilidad | Buena, pero más técnica | Muy alta |
| Manejo de errores | `.catch()` o segundo parámetro de `.then()` | `try...catch` |
| Ejecución paralela | `Promise.all()` | `await Promise.all()` |
| Ideal para | Procesos encadenados | Código con pasos dependientes |

---

## 5️⃣ Errores comunes

⚠️ **Olvidar el `.catch()` o el `try...catch`**
→ Resultado: “Unhandled Promise Rejection”.

⚠️ **Usar `await` fuera de una función `async`**
→ Error de sintaxis: *await is only valid in async functions*.

⚠️ **Encadenar mal los `.then()`**
→ Las promesas no se ejecutan en orden o no devuelven el resultado esperado.

💡 **Consejo práctico:**
Siempre devuelve la promesa (`return`) en cada paso de un `.then()`
si necesitas que el valor se pase al siguiente.

---

# 6️⃣ Patrón combinado `Promise.all()` + `async/await`

```
async function ejecutarTareas() {
  try {
    const [usuario, repos] = await Promise.all([
      fetch("https://api.github.com/users/octocat").then(r =>
r.json()),
      fetch("https://api.github.com/users/octocat/repos").then(r =>
r.json())
    ]);

    console.log("Usuario:", usuario.login);
    console.log("Repositorios:", repos.length);
  } catch (error) {
    console.error("Error:", error);
  }
}
```

📘 **Ventajas:**

- Ejecuta tareas en paralelo (más rápido).
- Mantiene el código legible y ordenado.
- Facilita el manejo de errores centralizado.

---

# 7️⃣ Buenas prácticas finales

✅ **Siempre captura los errores** con `.catch()` o `try...catch`.
✅ **Usa `await` solo donde sea necesario** (no en bucles grandes).
✅ **Combina `Promise.all()` para tareas paralelas.**
✅ **Documenta tus funciones asíncronas:** especifica qué devuelven y cuándo se resuelven.
✅ **Evita bloqueos:** recuerda que `await` pausa la función actual, no el hilo global.

---

# 8️⃣ Conclusión general

Las **Promesas** y **Async/Await** son herramientas esenciales para manejar la asincronía moderna en JavaScript.
Han reemplazado completamente a los callbacks tradicionales y permiten escribir código:

- Más limpio
- Más legible
- Más fácil de mantener
- Más seguro ante errores

💬 **Dominar este tema** te permitirá construir aplicaciones web eficientes, rápidas y con un control total sobre los procesos asíncronos.